Serverless Application Model

20/4/26 21:28

AWS Serverless Application Model

- **AWS Serverless Application Model (AWS SAM)** is an open-source framework for building serverless applications. It provides a shorthand syntax to express functions, APIs, databases, and event source mappings.
- It is an **extension of AWS CloudFormation**, meaning any resource declared in a SAM template is transformed into standard CloudFormation resources during deployment.
- You create a **JSON** or **YAML** configuration template to model your applications.
- During deployment, SAM transforms and expands the SAM syntax into **AWS CloudFormation syntax**. Any resource that you can declare in an AWS CloudFormation template you can also declare in an AWS SAM template.
- **AWS SAM Template:** An extension of CloudFormation that provides simplified syntax for defining serverless resources.
- **AWS SAM CLI:** Provides a Lambda-like execution environment that lets you locally build, test, and debug applications defined by SAM templates. You can also use the SAM CLI to deploy your applications to AWS.
- You can use AWS SAM to build serverless applications that use **any runtime supported by AWS Lambda**. You can also use SAM CLI to locally debug Lambda functions written in Node.js, Java, Python, and Go.
- Template Anatomy:
  - If you are writing an AWS Serverless Application Model template alone and not via CloudFormation, the **Transform** section is required.
  - The **Globals** section is unique to AWS SAM templates. It defines properties that are common to all your serverless functions and APIs. All the **AWS::Serverless::Function**, **AWS::Serverless::Api**, and **AWS::Serverless::SimpleTable** resources inherit the properties that are defined in the **Globals** section.
  - The **Resources** section can contain a combination of AWS CloudFormation resources and AWS SAM resources.

When to Use SAM

- Serverless applications
  - Quickly define AWS Lambda functions, Amazon API Gateway APIs, Amazon DynamoDB tables, and other serverless resources with minimal code.
- CloudFormation enhancement:
  - You can combine SAM with existing CloudFormation templates to add serverless components to traditional infrastructure. SAM resources work alongside standard CloudFormation resources in the same template.
- Local development and testing:
  - Use the SAM CLI to test Lambda functions locally, simulate API Gateway endpoints, and debug serverless applications on your development machine before deploying to AWS.
- CI/CD for serverless:
  - Build deployment pipelines using SAM templates that automatically generate the CloudFormation infrastructure needed for staging and production environments.
- Migration from console-created resources:
  - Convert Lambda functions and API Gateway resources created in the AWS Management Console into infrastructure as code using SAM templates.

Overview of Syntax

- AWS::Serverless::Api
  - This resource type describes an API Gateway resource. It's useful for advanced use cases where you want full control and flexibility when you configure your APIs.
- AWS::Serverless::Application
  - This resource type embeds a serverless application from the AWS Serverless Application Repository or from an Amazon S3 bucket as a nested application. Nested applications are deployed as nested stacks, which can contain multiple other resources.
- AWS::Serverless::Function
  - This resource type describes configuration information for creating a Lambda function. You can describe any event source that you want to attach to the Lambda function—such as Amazon S3, Amazon DynamoDB Streams, and Amazon Kinesis Data Streams.
- AWS::Serverless::LayerVersion
  - This resource type creates a Lambda layer version that contains library or runtime code needed by a Lambda function. When a serverless layer version is transformed, AWS SAM also transforms the logical ID of the resource so that old layer versions are not automatically deleted by AWS CloudFormation when the resource is updated.
- AWS::Serverless::SimpleTable
  - This resource type provides simple syntax for describing how to create DynamoDB tables.
- AWS::Serverless::HttpApi
  - Describes an API Gateway HTTP API (faster/cheaper than REST APIs).
- AWS::Serverless::StateMachine
  - Describes an AWS Step Functions state machine for workflows.
- AWS::Serverless::Connector
  - Simplifies IAM permissions by defining relationships between resources (e.g., Source: Function -> Destination: Table).

Commonly Used SAM CLI commands

- The sam init command generates pre-configured AWS SAM templates.
- The sam local command supports local invocation and testing of your Lambda functions and SAM-based serverless applications by executing your function code locally in a Lambda-like execution environment.
- The sam package and sam deploy commands let you bundle your application code and dependencies into a "deployment package" and then deploy your serverless application to the AWS Cloud.
- The sam logs command enables you to fetch, tail, and filter logs for Lambda functions.
- The output of the sam publish command includes a link to the AWS Serverless Application Repository directly to your application.
- Use sam validate to validate your SAM template.
- Use sam build command to prepare your application for deployment by resolving dependencies and organizing artifacts.
- The sam sync command watches for local changes and automatically syncs them to the AWS Cloud, bypassing full CloudFormation stacks to update code in seconds (AWS SAM Accelerate).

Controlling Access to APIs

- You can use AWS SAM to control who can access your API Gateway APIs by enabling authorization within your AWS SAM template.
  - A **Lambda authorizer** (formerly known as a custom authorizer) is a Lambda function that you provide to control access to your API. When your API is called, this Lambda function is invoked with a request context or an authorization token that is provided by the client application. The Lambda function returns a *policy document* that specifies the operations that the caller is authorized to perform, if any. There are two types of Lambda authorizers:
    - Token based type receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
    - Request parameter based type receives the caller's identity in a combination of headers, query string parameters, stageVariables, and $context variables.

  - **Amazon Cognito user pools** are user directories in Amazon Cognito. A client of your API must first sign a user in to the user pool and obtain an identity or access token for the user. Then your API is called with one of the returned tokens. The API call succeeds only if the required token is valid.

- The optional **Transform** section of a CloudFormation template specifies one or more macros that AWS CloudFormation uses to process your template. Aside from macros you create, AWS CloudFormation also supports the **AWS::Serverless** transform which is a macro hosted on AWS CloudFormation.
  - The AWS::Serverless transform specifies the version of the AWS Serverless Application Model (AWS SAM) to use. This model defines the AWS SAM syntax that you can use and how AWS CloudFormation processes it.

From <https://tutorialsdojo.com/aws-serverless-application-model-sam/>